NASHR نشر · DEVELOPER RESPONSE

How we will build Nashr

Prepared for Omar Siyam · 29 October 2026 · In reply to the Developer Brief of 28 October 2026

I have gone through the brief in detail and checked the prototype against it. This document sets out my recommendation on how to build Nashr, the languages we can build it in, the gaps I found in the brief (each with a suggested fix), and the decisions I need from you before I start.

In short

- **Rebuild everything from scratch with a full redesign**, using the prototype as the reference for how Nashr behaves.
- **Build it in TypeScript** (React for the screens, Node.js for the server) on a PostgreSQL database, with Stripe Connect for payments.
- **24 gaps in the brief** need a decision. I have suggested a fix for each one.
- **9 questions for you** are listed in section 5.

1. Recommendation: rebuild with a full redesign

The prototype is a good reference for how Nashr should behave. But it is one very large file with every rule (statuses, timers, fees, refunds, who sees what) running in the buyer's or seller's browser, and its data is stored in the browser too. It cannot be made safe or maintainable for a live product. As the brief says in section 25, all of these rules must run on the server.

I recommend:

- Rebuilding everything (buyer, seller, admin and founder) on a proper server.
- A full redesign of all screens in Figma first, for your approval before building.
- Keeping the prototype as the reference for every rule and flow, and turning its behaviour into automated tests.

2. What the prototype has, and what is missing

The buyer and seller sides are well developed. The admin and the founder reports are where most of the missing work is.

Admin pages (brief section 17)

Page	In prototype	What is there / what is missing
Overview	Built	Commission, GMV, take rate, net take after card fees, charts, "Needs your decision" list
Access requests	Built	Approve with invite link, reject with reason, request more information
Publishers	Built	Fee rate change with history, acceptance rate, response time. Missing Stripe payout status
Funds	Built	Money by publisher in five stages, reconciliation check, click through to deals
Accounts	Partial	List only. Missing verify, suspend, ban, resend invite, reset access, view activity
Outlet profiles	Partial	Create, edit, delete. Missing price ranges (confirmed/estimated) and change history. Deleting goes against "history must be kept"
All deals	Partial	Stage counts, filter by publisher. Missing status/market/channel/buyer/date filters, search, export, refund and cancel buttons, full timeline
Disputes	Partial	The four outcomes exist inside the decision list. No page of its own with both sides' evidence
Audit log	Partial	A plain list. No filter or export, and it is stored in the browser so it can be edited
Calendar oversight	Not built	Whole page
Payouts	Not built	Whole page
Messages and notifications	Not built	Whole page (templates, send log, resend)
Settings	Not built	Whole page. Timers and rates are currently fixed in the code
Reports	Not built	Whole page
Refund costs	Not built	Whole page

Missing across every admin page: CSV export, search, a separate founder login, two-factor login, and log-in-as-user for support.

Founder reports (brief section 18)

About 4 of the 13 reports are partly covered by the overview: revenue, GMV and take rate, money position, and quality and trust. **Still to build:** buyer activation, supply, demand, prices, top lists, retention, cross-border, failures, and disputes and refund losses, plus the date, market, channel, publisher and buyer filters, CSV export and the weekly email.

Other features not in the prototype

Quick sale and "Show interest", Report an issue, the proof checklist per channel, the 48-hour pre-run check, chargebacks and goodwill refunds, and the KSA "Coming soon" page with its waiting list. Athena is fully built in the prototype but, as the brief says, it will only be shown as "Coming soon".

3. Languages we can build on

Three languages can build Nashr properly. Because we are redesigning, the screens will be built in React whichever server language we choose. The question is what runs the server.

What matters	TypeScript	Python (Django)	PHP (Laravel)
Languages in the system	One for everything	Two Python + React	Two PHP + React
Screens and server in sync	Shared data defined once	Manual defined twice	Manual defined twice
Money and privacy safety	Strong typing	Good typing optional	Good weaker typing
Admin tables	Toolkit (Refine)	Built in (plain look)	Filament
Timers and live updates	In the database	Extra services (Celery, Redis)	Built in
Stripe Connect	Official SDK	Official SDK	Official SDK
Hiring in the UAE	Very easy	Easy	Easy

TypeScript (React + Node.js)

Recommended

`Node.js 22 · Fastify · PostgreSQL 16 · React · Refine (admin) · Stripe SDK`

STRENGTHS

- One language for server, buyer and seller screens, admin and reports.
- The data is defined once, so screens and server cannot fall out of step.
- Strong typing: a buyer can never be sent the reserve price or our fee rate.
- Best fit for a live calendar and bidding screens.
- Timers, bid locking and live updates run inside the database. Nothing extra to host.

WEAKNESSES

- No ready-made admin panel. Admin tables are built with a toolkit (Refine).
- More set-up choices at the start than an all-in-one framework.

Best if we want one modern, consistent product and the safest handling of money and permissions.

Python (Django)

Strong alternative

`Python 3.13 · Django 5 · PostgreSQL 16 · Celery + Redis · React · Stripe SDK`

STRENGTHS

- Admin tables (search, filters, edit, history) come almost free.
- Mature login, permissions and security protections built in.
- Best fit later for Athena and the reports.

WEAKNESSES

- Two languages, because the redesigned screens still need React.
- Timers and live updates need extra services to run and monitor.
- The built-in admin looks plain next to a full redesign.

Best if admin tooling and future data work matter more than having one language.

PHP (Laravel)

Workable

`PHP 8.3 · Laravel 11 · PostgreSQL 16 · Filament (admin) · Inertia + React · Stripe SDK`

STRENGTHS

- Timers, queues and email built in.
- Filament gives a polished admin quickly.
- Large developer pool in the region.

WEAKNESSES

- Two languages (PHP + React).
- Weaker typing, so money and privacy rules lean more on tests.
- Needs a proper server, not shared hosting.

Best if the team that maintains Nashr after launch is PHP-based.

The same whichever language we choose

- **PostgreSQL** as the database: it locks a slot so two bids can never both win, and keeps an audit log nobody can edit.
- **Stripe Connect** with separate charges and transfers: the buyer is charged once, Nashr holds the money, the seller is paid at release.
- **Amounts stored in fils and halalas** (whole numbers, never decimals): `76,000.00 AED` is stored as `7,600,000`.
- **Hosting in a UAE region**, with separate test and live systems and daily backups with a tested restore.
- **Every timer, rate and percentage is a setting**, so changes never need new code.

4. Gaps in the brief, and my suggested fixes

These are points the brief leaves open, where the brief and the prototype disagree, or where there is a risk. The suggested fix is what I will build unless you decide otherwise.

Money and tax

#	Gap	Suggested fix
1	**Card fees on big deals.** A 76,000 AED booking costs about 2,200 AED in card fees, roughly a quarter of our 9,120 fee, and many corporate cards cannot take that amount.	Accept bank transfer above a set amount.
2	**VAT on the ad itself** is not covered, only VAT on our fee. Buyers need a tax invoice for the media to reclaim VAT.	Accountant to confirm who issues it. Likely the publisher's invoice, generated by Nashr.
3	**Paying Saudi publishers** through Stripe is not confirmed.	Launch the UAE on Stripe. Confirm KSA with Stripe before opening, with a backup provider option.
4	**No limit on how long we hold money** before release or refund.	Release or refund within 90 days; admin warned at day 60. Confirm with Stripe.
5	**Refund after the seller was already paid**, when there are no future payouts to deduct from.	Track it as money owed, deduct from the next payouts, admin chases it after a set time.
6	**Cancellation fee maths**: the order of the 15% fee, our commission and the card cost is not defined.	One agreed formula, tested with real numbers.
7	**Exchange rate on SAR refunds** will not match the AED reports.	Record the difference separately in the ledger.

Booking rules

#	Gap	Suggested fix
8	**"Next bid moves up"**: the brief says the seller decides; the prototype offers it straight to the next buyer.	Back to the seller with a new 24 hours, at most twice, then relist.

#	Gap	Suggested fix
9	**Payment time "shortened if the creative deadline is close"** has no formula.	48 hours, or 24 hours before the creative deadline, whichever is earlier.
10	**Proof deadline** has no number.	Print: 3 working days after the run. Other channels: 2.
11	**Can a buyer withdraw a normal bid?** Not defined.	No, bids are binding. Admin can cancel one with a reason.
12	**Auto-bid ties** between two equal maximum bids.	The earlier maximum wins.
13	**Budget is only a warning**; a buyer can win more than their budget.	Show "total if you win everything" in the bid panel.
14	**UAE tax invoices** must be numbered in sequence.	Separate numbering per document type and market.
15	**Placements with no end date**: nothing creates future slots.	Keep 26 weeks of slots ahead automatically.
16	**Overnight slots** (for example 23:00 to 02:00): which date do they belong to?	The start date, in the outlet's time zone.
17	**After 2 rounds of creative changes**, what happens is not defined.	The seller must approve or reject with a reason.
18	**"Issue open" and "make-good scheduled"** pause timers but are not in the status list.	Add them as statuses.
19	**"Typical on Nashr" price** from only 1 or 2 sales reveals a buyer's exact price.	Show it only after 3 or more sales.

Security and operations

#	Gap	Suggested fix
20	**Blocking phone numbers in chat** is easy to get around (Arabic digits, spaced numbers).	Block, and flag doubtful messages for review. This reduces leakage; it will not stop all of it.
21	**Log-in-as-user** for support has no limits.	Read-only, always logged, no money actions.
22	**Data protection laws** (UAE and Saudi) are not mentioned.	Host in the UAE. Check the KSA rules before opening.
23	**Demo code** must never reach the live system.	Kept out of the new build entirely.
24	**Backups**: "tested restore" has no target.	At most 5 minutes of data lost, back up and running within 4 hours, restore tested before launch.

5. What I need from you

1. **Rebuild:** are you OK with a full rebuild and redesign?
2. **Language:** TypeScript, or do you prefer Python or PHP?
3. **Design:** do you have brand guidelines (logo, colours, fonts), or should the redesign create them?
4. **Large payments:** bank transfer for large bookings in the pilot, or card only?
5. **VAT:** can you ask the accountant who invoices the media sale and its VAT?
6. **Saudi payouts:** OK to launch the UAE on Stripe and confirm KSA with them later?
7. **Next bid:** after a decline or lapse, does the slot go back to the seller, or straight to the next buyer?
8. **Accounts:** whose AWS (hosting) and Stripe accounts will the system run on?
9. **Fixes:** are you happy with my suggested fixes in section 4, or do you want to change any?

Once these are answered, I will prepare a phased build plan with timeline and cost. Happy to get on a call and go through it together.